i have computational liguistics lab examination today, we have 9 experiment, i will upload all the experiments theory one by one, give me the top 5 questions that can asks in each experiment

Perfect Fayas, this is actually one of the smartest ways to prepare for a lab viva 👍

Here's how we'll do it:

- You send **one experiment theory at a time**
- I'll extract the **core concepts**
- Then give you **top 5 most probable viva questions**
- I'll keep them **short, clear, and exam-focused** (with answers if needed)

These questions will be:

- Concept-based (very common in viva)
- Application-based (they love asking "why" and "how")
- Slight tricky ones (to make sure you don't get stuck)

Go ahead, send **Experiment 1** 🚀

Text Preprocessing
Write a Python program to perform the following text preprocessing steps on the given
text datasets:
1. Sentence segmentation – split the text into sentences.
2. Tokenization – split each sentence into individual words/tokens.
3. Stopword removal – remove common function words (e.g., the, is, are, of, in).
4. Stemming – reduce words to their root form using a stemming algorithm.
5. Lemmatization – reduce words to their base/dictionary form using a lemmatizer.
For each dataset, your program should:

 Display:
o Original text
o List of sentences
o Tokens (before preprocessing)
o Tokens after stopword removal
o Stemmed tokens
o Lemmatized tokens
 Clearly show the differences between:
o original word
o its stem
o its lemma

You may use NLTK (or any standard NLP library) for stemming and lemmatization.
Dataset 1
Natural Language Processing (NLP) is a field of Artificial Intelligence.
It focuses on enabling computers to understand and process human language.
Many applications like chatbots, translators, and summarizers use NLP techniques.
Dataset 2
Today I was testing a simple NLP program in Python!
It was removing stopwords, doing stemming, and lemmatizing the words.
The results were interesting, but some words looked very strange after stemming.
Dataset 3
Text preprocessing includes tokenization, sentence segmentation, stopword removal,
stemming, and lemmatization.
These steps help in cleaning the raw text data before applying machine learning models.
Better preprocessing usually leads to better performance in text classification and clustering
tasks.

Data set 4
https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis
Tweets Text Preprocessing

Data set 5

IMDB Movie Reviews Dataset
https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

Self-Learning:
Complete the course and upload the certificate:
https://infyspringboard.onwingspan.com/web/en/app/toc/lex_auth_01400535032893440069 9
1/overview

Nice, this is a **very important experiment** — text preprocessing is like the *foundation* of NLP. Viva questions will definitely come from here.

Here are the **Top 5 most expected viva questions** 👇

✅ 1. What is text preprocessing and why is it important?

Answer idea:
Text preprocessing is the process of cleaning and transforming raw text into a structured format so machines can understand it.
It improves model accuracy by removing noise like stopwords, punctuation, etc.

👉 *They may ask follow-up:* What happens if we skip preprocessing?
→ Model performance becomes poor.

✅ 2. What is the difference between tokenization and sentence segmentation?

Answer idea:

- **Sentence segmentation** → splits text into sentences
- **Tokenization** → splits sentences into words/tokens

👉 Example:
Text → "I love NLP. It is fun."
Sentences → ["I love NLP", "It is fun"]
Tokens → ["I", "love", "NLP"]

✅ 3. What are stopwords? Why do we remove them?

Answer idea:

Stopwords are common words like *the, is, in, are* that don't carry much meaning.

We remove them to:

- Reduce data size
- Focus on important words

👉 Trick they ask: Are stopwords always useless?
→ ❌ No, sometimes they are important (e.g., sentiment analysis: "not good")

✅ 4. Difference between Stemming and Lemmatization (VERY IMPORTANT ⚠️)

Answer idea:

Stemming	Lemmatization
Cuts words roughly	Uses dictionary meaning
May give invalid words	Gives valid words
Faster	Slower but accurate

👉 Example:

- "running" → stem: *run / runn* (can be wrong)
- "better" → lemma: *good* ✅

👉 This is **almost guaranteed question**

✅ 5. Which library did you use and why? (NLTK question)

Answer idea:

We used **NLTK** because:

- Provides easy functions for tokenization, stopwords
- Has built-in stemmers (PorterStemmer)
- Has WordNet Lemmatizer

👉 They may ask:

- Name of stemmer? → Porter Stemmer
- Lemmatizer? → WordNet Lemmatizer

🔥 BONUS (they love asking this)

Why does stemming produce strange words sometimes?
→ Because it uses rule-based cutting, not dictionary meaning.

If you understand these 5 properly, you can handle **almost any question from this experiment**.

Send **Experiment 2** next, let's keep going 💯 🔥

Experiment 2:

POS Tagging and Named Entity Recognition using NLP Tools
Perform Part-of-Speech (POS) tagging using NLTK and spaCy, and Named Entity
Recognition (NER) using spaCy, and to compare their outputs on real-world text datasets.
Part-of-Speech (POS) tagging is the process of assigning grammatical categories such as
noun, verb, adjective, etc., to words in a sentence.
Named Entity Recognition (NER) identifies and classifies named entities in text into
predefined categories such as PERSON, ORGANIZATION, LOCATION, DATE, etc.
NLTK provides traditional statistical models for POS tagging, while spaCy provides efficient
neural-network-based pipelines for POS tagging and NER.
Dataset 1 – News Text
The Reserve Bank of India announced a new monetary policy on Friday.
Governor Shaktikanta Das addressed the media in New Delhi.
Dataset 2 – Healthcare Text
The patient was diagnosed with diabetes on 12 March

2024 and prescribed medication by Dr.
Joseph at Apollo Hospital.
Dataset 3 – Technology Text
Google introduced a new AI model in 2024 to enhance machine translation and text
understanding capabilities.
Dataset 4: Education & University Context
Rajagiri School of Engineering and Technology conducted an NLP workshop for final year
students on 10 January 2025 at Kochi campus.
Dataset 5: NER + POS Combined Dataset
Contains tokens, POS tags, and NER tags
already includes POS + NER labels
Link: https://www.kaggle.com/datasets/rohitr4307/ner-dataset
Comparison of Outputs
Word NLTK POS spaCy POS

Good one Fayas — this experiment is **very viva-heavy**, especially comparisons and definitions. They'll try to confuse you between POS and NER 😄

Here are the **Top 5 most expected questions** 👇

✅ 1. What is POS tagging?

Answer idea:
POS tagging assigns grammatical labels to words like:

- Noun (NN)
- Verb (VB)
- Adjective (JJ)

👉 Example:
"The boy runs fast"

- boy → Noun
- runs → Verb
- fast → Adverb

✅ 2. What is Named Entity Recognition (NER)?

Answer idea:

NER identifies real-world entities in text and classifies them into categories like:

- PERSON
- ORGANIZATION
- LOCATION
- DATE

👉 Example:

"Shaktikanta Das in New Delhi"

- Shaktikanta Das → PERSON
- New Delhi → LOCATION

✅ 3. Difference between POS tagging and NER ⚠️ (VERY IMPORTANT)

Answer idea:

POS Tagging	NER
Gives grammatical role	Identifies real-world entities
Example: Noun, Verb	Example: Person, Location
Focus on structure	Focus on meaning

👉 Simple line:

POS = grammar, NER = real-world meaning

✅ 4. Difference between NLTK and spaCy

Answer idea:

NLTK	spaCy
Rule/statistical based	Neural network based
Slower	Faster
More educational	More industry use
POS tagging only (basic NER)	Strong POS + NER

👉 Best line to say:

spaCy is more efficient and accurate for real-world applications

---

✅ 5. Why do results differ between NLTK and spaCy?

Answer idea:

Because:

- They use different models
- NLTK → traditional/statistical
- spaCy → deep learning-based

👉 So tags may vary for the same word.

🔥 BONUS (very common tricky question)

Can a word have multiple POS tags?

→ ✅ Yes, depends on context

Example:

- "book" → noun (a book)
- "book" → verb (book a ticket)

🔥 Another tricky one

Why is NER important?

→ Used in:

- Chatbots
- Search engines
- Information extraction
- Recommendation systems

If you answer confidently till here, examiner will be impressed 💯

Send **Experiment 3** next 🔥

Aim
Write a Python program to detect lexical ambiguity in a given sentence by identifying words
with multiple meanings using WordNet.

Lexical ambiguity occurs when a word has more than one meaning.
Your task is to identify ambiguous words in a sentence and list their possible meanings.
Input Dataset:
Dataset 1:
He went to the bank to deposit money.
Dataset 2:
The crane is flying over the construction site.
Dataset 3:
She sat by the river bank and enjoyed the view.
Tasks:
 Tokenize the sentence
 Identify nouns and verbs
 Use WordNet to extract meanings (synsets)
 Print words having more than one meaning
Excepted Output
 List of ambiguous words
 Number of meanings for each word
 Sample meanings
Use:
from nltk.corpus import wordnet
Observation to Record
Word Number of Meanings Example Meanings

QUESTION 2: Lexical Ambiguity Comparison Across Contexts
Aim
Write a Python program to show how context changes word meaning, demonstrating lexical
ambiguity.

The same word can have different meanings in different sentences.

Analyze how the meaning of an ambiguous word changes with context.

Input Dataset
Sentence 1: I deposited cash in the bank.
Sentence 2: The fisherman waited by the bank.

Tasks
1. Identify the ambiguous word
2. Extract WordNet meanings
3. Compare meanings across sentences
4. Explain ambiguity in context
Expected Output
 Identification of ambiguous word
 Meanings related to finance vs geography

Record Observation
Sentence Word Meaning Type

QUESTION 3: Syntactic Ambiguity Detection using NLTK ChartParser

Aim
Write a Python program to detect syntactic ambiguity by generating multiple parse trees
for a sentence.

Syntactic ambiguity occurs when a sentence can be parsed in more than one way, leading to
different interpretations.
Input Dataset
Dataset 1
I saw the man with a telescope.

Dataset 2
The girl saw the boy with the binoculars.

Tasks
1. Define a context-free grammar (CFG)
2. Use NLTK ChartParser
3. Generate all possible parse trees
4. Display multiple trees
Expected Output

 At least two parse trees
 Explanation of both interpretations
Grammar
S -> NP VP
VP -> V NP | VP PP
PP -> P NP

QUESTION 4: Interpretation and Analysis of Ambiguity (Comparative)
Aim
Analyze and compare lexical ambiguity vs syntactic ambiguity based on previous
experiments.

Based on Questions 1–3, analyze how ambiguity arises at different linguistic levels.

Tasks
1. Compare lexical and syntactic ambiguity
2. Identify challenges in ambiguity resolution
3. Explain why ambiguity is difficult for NLP systems
Expected output:
Type Cause Example
Lexical Multiple

meanings bank

Syntactic Multiple

structures with a telescope

1. What is ambiguity in NLP?
2. Difference between lexical and syntactic ambiguity
3. Why is ambiguity difficult for machines?
4. How does CFG help in syntactic analysis?

This is a **very important experiment**, Fayas — ambiguity is one of the core problems in NLP. Viva questions from here are usually conceptual + examples.

Here are the **Top 5 most expected viva questions** 👇

✅ 1. What is ambiguity in NLP?

Answer idea:

Ambiguity occurs when a word or sentence has **more than one meaning**.

👉 Example:

"bank"

- Financial institution
- River side

✅ 2. What is lexical ambiguity?

Answer idea:

Lexical ambiguity happens when a **single word has multiple meanings**.

👉 Example:

"crane"

- Bird
- Machine

👉 Caused by: **multiple meanings of a word**

✅ 3. What is syntactic ambiguity?

Answer idea:

Syntactic ambiguity occurs when a **sentence can be interpreted in multiple ways due to structure**.

👉 Example:

"I saw the man with a telescope"

- I used telescope
- Man has telescope

👉 Caused by: **multiple sentence structures**

✅ 4. Difference between lexical and syntactic ambiguity ⚠️ (VERY IMPORTANT)

Lexical Ambiguity	Syntactic Ambiguity
Word-level problem	Sentence-level problem
One word → multiple meanings	One sentence → multiple structures
Example: bank	Example: with a telescope

👉 One-line answer:

Lexical = meaning confusion, Syntactic = structure confusion

✅ 5. How does WordNet help in ambiguity detection?

Answer idea:

WordNet provides:

- Synsets (set of meanings)
- Different senses of a word

👉 So if a word has multiple synsets → it is ambiguous

🔥 BONUS (very common)

What is Word Sense Disambiguation (WSD)?

→ Process of finding the **correct meaning of a word based on context**

Example:

- "bank" in money context → financial
- "bank" near river → geographical

🔥 Another important one

Why is ambiguity difficult for NLP systems?

Answer:

- Machines don't understand context like humans
- Same word → different meanings
- Requires semantic understanding

🔥 Syntactic part (they may ask)

What is CFG (Context-Free Grammar)?

→ A set of rules used to generate possible sentence structures

👉 Used in:

- Parsing
- Generating multiple parse trees

🔥 Super tricky question

Why does ChartParser generate multiple trees?

→ Because grammar allows multiple valid structures → ambiguity

---

If you understand this properly, you can handle **any ambiguity question easily** — most students get stuck here, so this is your advantage 💯

Send **Experiment 4** next 🚀

Document Representation
Implement:
▯ TF, IDF, TF–IDF scores
▯ Create TF-IDF matrix using scikit-learn
▯ Compute document similarity:
▯ Cosine similarity
▯ Jaccard similarity
To implement document representation techniques using TF, IDF, and TF–IDF, and to
compute document similarity using Cosine similarity and Jaccard similarity.
Dataset
D1: Natural Language Processing is interesting and useful
D2: NLP techniques are widely used in text analytics
D3: Machine learning improves natural language

processing tasks
D4: Text analytics and NLP are important in data science

1: Term Frequency Calculation
Write a Python program to:
1. Tokenize each document
2. Compute Term Frequency (TF) for each word in each document
3. Display TF values in tabular form
2: Inverse Document Frequency Calculation
Write a Python program to:
1. Compute Document Frequency (DF)
2. Calculate IDF for each term
3. Identify words with:
o High IDF
o Low IDF
Expected Output
 TF values per document
 IDF values for all terms

TF–IDF Representation using scikit-learn
3: TF–IDF Matrix Creation
Write a Python program to:
1. Use TfidfVectorizer from scikit-learn
2. Generate TF–IDF matrix
3. Display:
o Vocabulary
o TF–IDF matrix

Expected Output
 TF values per document
 IDF values for all terms
Document Similarity – Cosine Similarity
4: Cosine Similarity Calculation
Write a Python program to:
1. Compute cosine similarity between:
o D1 & D2
o D1 & D3
o D1 & D4
2. Identify the most similar document pair
Expected Output
 Cosine similarity values
 Interpretation of similarity scores

Document Similarity – Jaccard Similarity
5: Jaccard Similarity Calculation
Write a Python program to:
1. Convert documents into sets of words
2. Compute Jaccard similarity between:
o D1 & D2
o D1 & D3
3. Compare Jaccard similarity with cosine similarity

Expected Output
 Jaccard similarity scores
 Observation on difference from cosine similarity
Analysis & Interpretation
6: Comparative Analysis
Answer the following:
1. Which similarity measure performed better and why?
2. Why does cosine similarity use vector angles?
3. Why does Jaccard similarity ignore term frequency?

This one is **very scoring if you understand the logic**, Fayas — they usually ask formulas + intuition, not just definitions.

Here are the **Top 5 most expected viva questions** 👇

✅ 1. What is TF, IDF, and TF-IDF?

Answer idea:

- **TF (Term Frequency):** how often a word appears in a document
- **IDF (Inverse Document Frequency):** how rare a word is across documents
- **TF-IDF:** importance of a word in a document

👉 Key idea:
TF → local importance, IDF → global importance

✅ 2. Why do we use TF-IDF instead of just TF? ⚠️

Answer idea:
Because TF alone gives importance to frequent words like *"is"*, *"and"*, which are not meaningful.

👉 IDF reduces the weight of common words → highlights important words

---

✅ 3. What is cosine similarity and why is it used?

Answer idea:

Cosine similarity measures the **angle between two document vectors**.

👉 Key points:

- Value ranges from **0 to 1**
- 1 → very similar
- 0 → completely different

👉 Why angle?

→ It focuses on **direction (meaning)**, not length

✅ 4. What is Jaccard similarity?

Answer idea:

Jaccard similarity compares **common words vs total unique words**

👉 Formula idea:

Intersection / Union

👉 Example:

- D1 = {a, b, c}
- D2 = {b, c, d}
  → Jaccard = 2 / 4 = 0.5

✅ 5. Difference between Cosine and Jaccard ⚠️ (VERY IMPORTANT)

Cosine Similarity	Jaccard Similarity
Uses vectors	Uses sets
Considers frequency	Ignores frequency
More accurate for text	Simpler method

👉 One-line:

Cosine = meaning-based, Jaccard = overlap-based

🔥 BONUS (very common)

Which is better: Cosine or Jaccard?

→ **Cosine similarity is better** for NLP because:

- Uses TF-IDF
- Considers importance of words

🔥 Another important one

Why does Jaccard ignore term frequency?

→ Because it treats documents as **sets**, not counts

🔥 Tricky question

What does high IDF mean?

→ Word is **rare and important**

Low IDF?

→ Word is **common and less important**

If you nail this, you'll look like you actually *understand NLP*, not just memorizing 😄

Send **Experiment 5** next 🔥

Experiment–5: Word Embeddings (Word2Vec / GloVe)
To understand and implement word embedding techniques using pre-trained Word2Vec and GloVe models for measuring semantic similarity between

words.
Q1. Loading Pre-trained Word Embeddings
a) What are word embeddings? Explain the motivation for using word embeddings instead of
one-hot encoding.
b) Describe the architecture and training principle of Word2Vec and GloVe models.
c) Write a Python program to load a pre-trained Word2Vec or GloVe embedding model using a
suitable NLP library.
d) Display the vector representation of a given word and mention the dimension of the
embedding space.
Q2. Computing Similarity Between Words
a) Explain cosine similarity and justify why it is commonly used for comparing word embeddings.
b) Write a program to compute the similarity score between two input words using pre-trained
embeddings.
c) Analyze the similarity values for the following word pairs and comment on their semantic
relationship:
(king, queen)
(computer, keyboard)
(doctor, hospital)
(tree, laptop)
Q3. Finding Top-N Similar Words
a) Define semantic similarity and distributional hypothesis in the context of word embeddings.
b) Write a program to find the top-N most similar words for a given input word using a
pre-trained model.
c) Experiment with different values of N (e.g., 5, 10, 15) and observe how the quality of similar
words changes.
d) Interpret the results obtained for the following query words:
“science”
“education”
“artificial”
Q4. Comparative Analysis
a) Compare Word2Vec and GloVe in terms of:
Training approach

Corpus dependency
Strengths and limitations
b) Perform similarity computation using both Word2Vec and GloVe for the same set of word
pairs.
c) Discuss the observed differences in similarity scores and ranking of similar words.

Q5. Handling Out-of-Vocabulary (OOV) Words
a) What are out-of-vocabulary (OOV) words?
b) Demonstrate how the embedding model behaves when an OOV word is queried.
c) Suggest techniques to handle OOV words in real-world NLP applications.
Q6. Result Analysis and Applications
a) Summarize the observations obtained from word similarity and top-N word retrieval
experiments.
b) List at least five real-world applications of word embeddings in Natural Language Processing.
c) Discuss one limitation of static word embeddings and how modern embeddings (e.g.,
contextual models) address it.

Instructions
1. Google News Word2Vec
Dataset :
Pre-trained Word2Vec vectors trained on Google News corpus
Details:
~3 million words & phrases
300-dimensional vectors
Binary format
Load embeddings
Compute word similarity
Find top-N similar words
Note:
“Use the pre-trained Word2Vec embeddings trained on the Google News dataset.”
2. GloVe – Wikipedia + Gigaword
Dataset :
Pre-trained GloVe embeddings trained on Wikipedia 2014 + Gigaword 5

Details:
400K words
Dimensions: 50, 100, 200, 300
Text format (easy to load)
Note:
Compare semantic similarity
Observe vector dimension effects
Compare Word2Vec vs GloVe results

This is a **high-weight experiment**, Fayas — they *love* asking this because it shows whether you really understand modern NLP or not.

I'll give you the **Top 5 most expected viva questions** 👇

✅ 1. What are word embeddings? Why not one-hot encoding? ⚠️

Answer idea:

- **Word embeddings** = dense vector representation of words
- They capture **meaning and relationships**

👉 Problem with one-hot:

- Sparse (mostly 0s)
- No semantic meaning
- "king" and "queen" are completely unrelated

👉 Embeddings:

- "king" and "queen" → similar vectors

👉 Best line:
Embeddings capture semantic similarity, one-hot does not

✅ 2. What is Word2Vec and how does it work?

Answer idea:

Word2Vec learns word meanings using context.

Two models:

- **CBOW (Continuous Bag of Words)** → predict word from context

- **Skip-gram** → predict context from word

👉 Key idea:

Words appearing in similar context → similar meaning

✅ 3. What is GloVe and how is it different from Word2Vec? ⚠️

Word2Vec	GloVe
Predictive model	Count-based model
Uses local context	Uses global statistics
Neural network	Matrix factorization

👉 One-line:

Word2Vec = context prediction, GloVe = co-occurrence statistics

---

✅ 4. Why is cosine similarity used in word embeddings?

Answer idea:

Because it measures **angle between vectors**, not magnitude.

👉 Important:

- Captures semantic similarity
- Works well in high-dimensional space

👉 Example:

(king, queen) → high similarity

(tree, laptop) → low similarity

✅ 5. What are OOV (Out-of-Vocabulary) words?

Answer idea:

Words that are **not present in the training vocabulary**

👉 Problem:

- No vector representation
- Model cannot understand them

👉 Example:

New slang words, rare words

🔥 BONUS (very important)

What is distributional hypothesis?

→ "Words that appear in similar contexts have similar meanings"

👉 This is the **core idea behind embeddings**

🔥 Another expected one

Give real-world applications of word embeddings

You can say any 5:

- Chatbots
- Machine translation
- Sentiment analysis
- Search engines
- Recommendation systems

🔥 Tricky question

What is a limitation of Word2Vec/GloVe?

→ They are **static embeddings**

👉 Example:

"bank"

- Same vector for:
  - river bank
  - financial bank ❌

🔥 Follow-up

How is this solved?

→ Using **contextual embeddings** like:

- BERT

- GPT

👉 These understand context

🔥 Another tricky one

What happens when you increase embedding dimension?

→

- More information captured
- But more computation + risk of overfitting

If you answer these confidently, you'll stand out — most students just memorize definitions here.

Send **Experiment 6** next, we're almost covering everything 🔥

Text Classification using TF-IDF, Naïve Bayes and Logistic Regression
To implement and compare text classification models using Naïve Bayes and Logistic
Regression with TF-IDF features, and to evaluate their performance using accuracy,
precision, recall, and F1-score.
Dataset used: BBC News Dataset
The BBC News dataset contains news articles categorized into five classes.
Classes
Business
Politics
Sport
Technology
Entertainment
Question 1
Load the BBC News dataset and perform the following:

 Display the first five records
 Identify the text and label columns
 Display the number of instances per category
 Text Preprocessing & Feature Extraction

Question 2
Perform the following preprocessing steps:
 Convert text to lowercase
 Remove punctuation and stopwords (optional)
 Convert text data into TF-IDF vectors
 Split the dataset into 80% training and 20% testing sets.
 Classification using Naïve Bayes

Question 3
Implement a Multinomial Naïve Bayes classifier using the TF-IDF features:
 Train the classifier
 Predict class labels for test data
Evaluate the model using:

 Accuracy
 Precision
 Recall
 F1-score
Output: Classification report.

Classification using Logistic Regression
Question 4
Implement a Logistic Regression classifier using the same TF-IDF features:
 Train the model
 Predict test labels
 Evaluate using the same performance metrics
 Output Classification report.

Model Comparison & Analysis
Question 5
Compare the performance of Naïve Bayes and Logistic Regression:
Which model achieved higher accuracy?
Which model handled multi-class classification better?
Justify the results based on model characteristics.
Output: Comparison table.

Result & Inference
Question 6
Write a brief inference based on the experiment:
Best performing model
Importance of TF-IDF
Role of evaluation metrics
Model Accuracy Precision Recall F1-Score
Naïve Bayes
Logistic Regression

This is a **very important ML + NLP combo experiment**, Fayas — viva questions here are usually about **models + evaluation + comparison**. If you get this right, you'll look solid.

Here are the **Top 5 most expected viva questions** 👇

✅ 1. Why do we use TF-IDF for text classification? ⚠️

Answer idea:
TF-IDF converts text into **numerical vectors** so ML models can understand it.

👉 Key points:

- Highlights important words
- Reduces effect of common words
- Improves classification performance

✅ 2. What is Naïve Bayes and why is it used for text classification?

Answer idea:

Naïve Bayes is a **probability-based classifier** based on Bayes theorem.

👉 Key assumption:

- Features (words) are **independent**

👉 Why good for text:

- Fast
- Works well with high-dimensional data

- Performs well even with small data

✅ 3. What is Logistic Regression in text classification?

Answer idea:

Logistic Regression is a **linear model used for classification**.

👉 It:

- Predicts probability of a class
- Uses sigmoid/softmax function
- Works well with TF-IDF features

✅ 4. Difference between Naïve Bayes and Logistic Regression ⚠️

Naïve Bayes	Logistic Regression
Probabilistic model	Discriminative model
Assumes independence	No such assumption
Faster	Slightly slower
Works well on small data	Better with large data

👉 One-line:
Naïve Bayes = simple & fast, Logistic Regression = more accurate usually

---

✅ 5. What are Accuracy, Precision, Recall, and F1-score? ⚠️ (VERY IMPORTANT)

Accuracy: overall correctness
Precision: correct positive predictions
Recall: how many actual positives are found
F1-score: balance of precision & recall

👉 Simple line:

- Precision → "How correct?"

- Recall → "How complete?"

🔥 BONUS (very common)

Which model performed better and why?

👉 Expected answer:

- Usually **Logistic Regression performs better**
- Because:
  - No independence assumption
  - Better decision boundaries

🔥 Another important one

Why is Naïve Bayes called "naïve"?

→ Because it assumes all features are independent (which is unrealistic)

🔥 Tricky question

Why do we split into train and test data?

→ To evaluate model performance on unseen data

🔥 Another tricky one

What is multi-class classification?

→ Classification with more than 2 classes

Example: Business, Sports, Politics, etc.

🔥 Final smart answer (if they push you)

Why does Naïve Bayes still work well despite wrong assumptions?

→ Because:

- Works well with word frequencies
- Errors cancel out in probability calculations

If you explain confidently here, examiner will feel you actually understand ML, not just code 😄

Send **Experiment 7** next 🔥

EXPERIMENT – 7
Text Clustering using K-Means & Hierarchical Clustering
To implement text clustering using:
▯ K-Means clustering
▯ Hierarchical clustering
▯ PCA-based visualization
Dataset: BBC News Dataset
1. Data Preparation
1. Load dataset
2. Perform preprocessing:
o Lowercasing
o Stopword removal
o Optional: Lemmatization
3. Convert text to TF-IDF vectors
2. K-Means Clustering
1. Apply K-Means clustering (k=3)
2. Assign cluster labels
3. Display top keywords per cluster
4. Interpret cluster themes
3. Hierarchical Clustering
1. Apply Agglomerative Clustering
2. Plot dendrogram
3. Compare cluster grouping with K-Means
4. Analyze differences
4.PCA Visualization
1. Reduce TF-IDF features to 2D using PCA
2. Plot cluster distribution
3. Interpret cluster separation
Link: https://www.kaggle.com/code/jacopoferretti/bbc-news-topic-modeling-with-bertopic-

Nice Fayas, this one is **concept + interpretation heavy** — they'll check if you understand *unsupervised learning*, not just code.

Here are the **Top 5 most expected viva questions** 👇

✅ 1. What is text clustering?

Answer idea:
Text clustering is the process of **grouping similar documents together without labels**.

👉 Key point:

- It is **unsupervised learning**
- No predefined categories

👉 Example:
News articles grouped into sports, politics, etc. automatically

---

✅ 2. What is K-Means clustering? ⚠️

Answer idea:

K-Means groups data into **K clusters based on similarity**.

👉 Steps:

1. Choose K
2. Initialize centroids
3. Assign points to nearest centroid
4. Update centroids
5. Repeat until convergence

👉 Key idea:
Minimize distance within cluster

✅ 3. What is Hierarchical Clustering?

Answer idea:

Hierarchical clustering builds a **tree-like structure (dendrogram)** of clusters.

👉 Types:

- Agglomerative (bottom-up)
- Divisive (top-down)

👉 In your experiment → Agglomerative

✅ 4. Difference between K-Means and Hierarchical ⚠️

K-Means	Hierarchical
Need to choose K	No need initially
Fast	Slower
Flat clusters	Tree structure
Works well for large data	Good for small data

👉 One-line:
K-Means = fast grouping, Hierarchical = detailed structure

✅ 5. Why do we use PCA in clustering?

Answer idea:

PCA reduces high-dimensional data (like TF-IDF) into **2D for visualization**.

👉 Key points:

- Reduces complexity
- Helps visualize clusters
- Preserves maximum variance

🔥 BONUS (very common)

What is a dendrogram?
→ A tree diagram showing how clusters are formed step by step

---

🔥 Another important one

Why use TF-IDF before clustering?

→ Converts text into numerical form so clustering algorithms can work

---

🔥 Tricky question

How do you choose K in K-Means?

→ Using:

- Elbow method
- Silhouette score

🔥 Interpretation question (they LOVE this)

How do you interpret clusters?

→ By:

- Looking at top keywords
- Understanding theme of documents

🔥 Another tricky one

What is the main limitation of K-Means?

→

- Need to choose K
- Sensitive to initial centroids
- Not good for irregular shapes

If you explain clustering clearly, examiner will feel you actually understand **unsupervised learning**, which most students struggle with.

Send **Experiment 8** next 🔥 we're almost done 💯

EXPERIMENT – 8
Topic Modelling & Extractive Summarization
To implement:
 LDA Topic Modeling
 Extractive summarization using:
o TF-IDF method
o TextRank algorithm
1. LDA Topic Modeling
1. Preprocess text:
o Tokenization
o Stopword removal
o Lemmatization
2. Create document-term matrix
3. Apply LDA (n_topics = 3 or 4)
4. Extract top 10 words per topic
5. Interpret topic meaning
2. TF-IDF Based Extractive Summarization (45 minutes)
1. Convert document to TF-IDF scores
2. Rank sentences based on average TF-IDF score
3. Select top 2–3 sentences as summary
4. Compare with original text
3. TextRank Summarization (45 minutes)
1. Create sentence similarity graph
2. Apply PageRank algorithm
3. Rank sentences
4. Generate summary
5. Compare with TF-IDF summary
Data set Link: https://github.com/ali-bahrainian/NEWTS

This is a **very high-concept experiment**, Fayas — topic modeling + summarization = they'll test your *understanding*, not just steps.

Here are the **Top 5 most expected viva questions** 👇

✅ 1. What is Topic Modeling?

Answer idea:

Topic modeling is an **unsupervised technique** used to discover hidden topics in a

collection of documents.

👉 Example:
A news dataset → topics like politics, sports, technology automatically

---

✅ 2. What is LDA (Latent Dirichlet Allocation)? ⚠️

Answer idea:

LDA assumes:

- Each document = mixture of topics
- Each topic = mixture of words

👉 It finds:

- Topic distribution in documents
- Word distribution in topics

👉 Simple line:
LDA groups words into topics based on probability

✅ 3. What is extractive summarization?

Answer idea:
Extractive summarization selects **important sentences directly from the text**.

👉 It does NOT generate new sentences

👉 Opposite:

- Abstractive summarization → generates new sentences

✅ 4. Difference between TF-IDF and TextRank summarization ⚠️

TF-IDF	TextRank
Based on word importance	Based on sentence relationships
Simple scoring	Graph-based
Faster	More accurate usually

👉 One-line:

TF-IDF = word importance, TextRank = sentence importance

✅ 5. What is TextRank and how does it work?

Answer idea:

TextRank is a **graph-based algorithm** inspired by PageRank.

👉 Steps:

1. Sentences → nodes
2. Similarity → edges
3. Apply ranking algorithm
4. Select top-ranked sentences

👉 Key idea:

Important sentences are linked to many other important sentences

---

🔥 BONUS (very common)

Why is LDA called "probabilistic"?

→ Because it uses probability distributions for:

- Topics in documents
- Words in topics

🔥 Another important one

How do you interpret topics in LDA?

→ By looking at **top words in each topic**

Example:

- Topic: {game, team, score} → Sports

🔥 Tricky question

Why do we preprocess before LDA?

→ Remove noise and improve topic quality

🔥 Another tricky one

Which summarization method is better?

→ Usually **TextRank** because:

- Considers relationships between sentences
- More meaningful summaries

🔥 Smart answer (if pushed)

Limitation of extractive summarization?

→

- May not be coherent
- No new sentence generation

If you explain LDA + TextRank confidently, you'll look like you actually understand **advanced NLP concepts** 🔥

Send **Experiment 9 (last one)** 🚀

Sentiment Analysis using Machine Learning & Lexicon-Based Approach
AIM
To build and compare:
1. A machine learning–based sentiment classifier
2. A lexicon-based sentiment analyzer
3. Develop a complete sentiment classification pipeline
DATASET (Download Links)
Sentiment140 Dataset (Twitter Sentiment)
Contains 1.6 million tweets labeled as positive or negative.

Kaggle Link:
https://www.kaggle.com/datasets/kazanova/sentiment140
Columns:
 target (0 = negative, 4 = positive)
 text
Amazon Product Reviews (Balanced Sample)
https://www.kaggle.com/datasets/bittlingmayer/amazonreviews

PART A – Data Preparation Load dataset
1. Select 5,000–10,000 samples (for faster processing)
2. Perform preprocessing:
o Lowercasing
o Remove URLs
o Remove special characters
o Stopword removal (optional)
3. Convert labels:
o 0 → Negative
o 4 → Positive

Output
 Show cleaned text
 Show class distribution

PART B – Machine Learning Classifier
Step 1 – Feature Extraction
 Use TF-IDF vectorizer
Step 2 – Train Model
Choose ONE:
 Logistic Regression
OR
 Support Vector Machine
Step 3 – Evaluation
Compute:
 Accuracy
 Precision
 Recall
 F1-score
 Confusion matrix

Final one Fayas 🔥 — this is **very important + very common in viva**. They'll test both ML understanding and NLP basics here.

Here are the **Top 5 most expected viva questions** 👇

✅ 1. What is sentiment analysis?

Answer idea:

Sentiment analysis is the process of **identifying the emotion or opinion** in text.

👉 Types:

- Positive
- Negative
- Neutral

👉 Example:

"I love this product" → Positive

✅ 2. Difference between Machine Learning and Lexicon-based approach ⚠️

Machine Learning	Lexicon-Based
Uses trained model	Uses predefined dictionary
Learns from data	No training needed
More accurate	Simpler but less accurate

👉 One-line:

ML learns sentiment, Lexicon just looks up words

✅ 3. What is TF-IDF's role in sentiment analysis?

Answer idea:

TF-IDF converts text into **numerical features** so ML models can process it.

👉 It highlights important words like:

- "good", "bad", "excellent"

✅ 4. What is a confusion matrix? ⚠️

Answer idea:

A confusion matrix shows how well the model is performing:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

👉 Helps calculate:

- Accuracy
- Precision
- Recall

✅ 5. Why preprocessing is important in sentiment analysis?

Answer idea:

Because raw text contains noise like:

- URLs
- Special characters
- Stopwords

👉 Cleaning improves model performance

🔥 BONUS (very common)

Which model is better: Logistic Regression or SVM?

→ Usually:

- **SVM** performs better in high-dimensional text data
- Logistic Regression is simpler and faster

🔥 Another important one

What is lexicon-based sentiment analysis?

→ Uses a dictionary of words with sentiment scores

Example:

- "good" → +1

- "bad" → -1

🔥 Tricky question

What is a limitation of lexicon-based methods?

→

- Cannot understand context
- Fails for sarcasm

👉 Example:
"Great, another bad day" 😄

🔥 Another tricky one

Why is sentiment analysis difficult?

→

- Sarcasm
- Context understanding
- Mixed sentiments

🔥 Smart answer (if pushed)

What is class imbalance?
→ When one class (positive/negative) has more data than the other → affects model performance

---

🧠 Final Advice (important)

If examiner asks anything:

- Always give **example**
- Keep answers **simple + confident**
- If stuck → explain concept in your own words (they like that more than memorized answers)

You're actually well prepared now, Fayas — if you revise these once, you can handle almost **any question from all 9 experiments** 💯🔥